

MINI PROJECT REPORT

Smart Parking Reservation System

[Browse](#) • [Reserve](#) • [Pay](#) • [Cancel](#) • [Manage](#)

Submitted by

Team Member	Register Number
Sanjana M	MS254239
Shakthi	MS254231
Vinisha	MS254252

Academic Year 2026

Contents

1	Aim and Objective
2	Introduction
3	Technology Stack
4	Key Features
5	System Workflow
6	Individual Contributions
7	Code Excerpts
8	Screenshots
9	Conclusion

1. Aim and Objective

The aim of this project is to design and develop a **web-based Smart Parking Reservation System** that displays available parking spaces in real time and lets users reserve, pay for, cancel and manage parking slots online. The system also gives administrators a dashboard to manage parking locations, slot capacity and pricing.

Objectives

- Show users the available parking locations with live slot counts and hourly rates.
- Allow a user to reserve a slot for a chosen vehicle and time window, with an automatically calculated fee.
- Provide a simulated payment checkout with input validation and a transaction receipt.
- Let users view their reservations and cancel them, with a simulated refund.
- Provide role-based access so that only admins can add, edit or delete locations and view all reservations.

2. Introduction

Finding a parking space in busy areas of a city such as Bengaluru is time-consuming and leads to traffic congestion. Drivers often circle for long periods without knowing whether a slot is free, while parking operators have no simple way to track occupancy. The Smart Parking Reservation System solves this by moving the process online: the driver checks availability, books a slot in advance, pays, and receives a confirmation, while the operator controls locations and capacity from an admin panel.

The application has two user roles. A **regular user** signs up with a name, email and password. An **admin** signs up with an additional admin code meant only for parking staff. Payment is a demo simulation only; no real card data is processed or stored.

3. Technology Stack

Layer	Technology	Purpose
Front end	HTML5, CSS3, JavaScript	Responsive pages: login/sign-up, locations, checkout, reservations, admin dashboard
Back end	Node.js with Express (REST API, port 4000)	Routing, authentication, reservation and payment logic, admin endpoints
Data storage	JSON / lightweight database on the server	Users, parking locations, reservations and payment records
Authentication	Email + password, role flag (user / admin)	Session handling and role-based access to the admin page
Tools	VS Code, Git/GitHub, browser dev tools	Development, version control and testing

Note: the server runs locally at `localhost:4000`.

4. Key Features

User features

- Secure sign-up and login (minimum 6-character password); optional admin code for staff.
- Browse available parking locations with address, slots available (e.g. 10 / 10) and price per hour.
- Reserve & Pay: choose vehicle number, start time and end time; duration and total fee are computed automatically.
- Card form with live card preview, plus validation such as the *Expiry must be in MM/YY format* check.
- Payment receipt with transaction ID, location, vehicle, masked card (•••• 4432) and amount paid.
- My Reservations page with an active / cancelled status badge and a Cancel & refund option with confirmation.

Admin features

- Add a new parking location (name, address, total slots, price per hour).
- Edit or delete existing locations; view current availability of each location.
- View all reservations made by users, and revenue information on the dashboard.

5. System Workflow

Step	Actor	Action
1	User	Creates an account or logs in.
2	User	Views available parking locations and clicks Reserve & Pay .
3	System	Calculates duration and total: hours × rate (e.g. 2 h × ₹50 = ₹100).
4	User	Enters vehicle number and card details; the form is validated before submission.
5	System	Creates a reservation, reduces available slots by one, and returns a transaction ID.
6	User	Views the booking under My Reservations; may cancel it.
7	System	On cancellation, marks it cancelled, restores the slot and records a simulated refund.
8	Admin	Adds, edits or deletes locations and monitors all reservations.

6. Individual Contributions

Team Member	Register No.	Focus Area	Contribution
Sanjana M	MS254239	Admin dashboard polish	Location management (add, edit, delete), availability display, reservations overview and revenue display for admins.
Shakthi	MS254231	Locations / browse page and Checkout / payment flow polish	Available-locations cards with slot count and price, checkout form with automatic duration and total, card preview, expiry/format validation and payment success receipt.
Vinisha	MS254252	My Reservations polish	Reservation cards with status badges, payment and transaction details, cancel with confirmation dialog and refund status display.

All three members took part in planning, testing and integrating the modules into the final application.

7. Code Excerpts

The following excerpts illustrate the core logic of the system (Node.js / Express back end and JavaScript front end).

7.1 Admin: adding a parking location (server)

```
// routes/admin.js
router.post("/locations", requireAdmin, (req, res) => {
  const { name, address, totalSlots, pricePerHour } = req.body;
  if (!name || !address || totalSlots < 1 || pricePerHour < 0)
    return res.status(400).json({ error: "Invalid location details" });

  const location = {
    id: Date.now().toString(),
    name, address,
    totalSlots: Number(totalSlots),
    availableSlots: Number(totalSlots),
    pricePerHour: Number(pricePerHour),
  };
  db.locations.push(location);
  res.status(201).json(location);
});
```

7.2 Checkout: duration, total and expiry validation (client)

```
// checkout.js
function calcTotal(start, end, rate) {
  const hours = Math.ceil((new Date(end) - new Date(start)) / 36e5);
  return { hours, total: hours * rate }; // 2 h x 50 = 100
}

function validateExpiry(value) {
  if (!/^(0[1-9]|1[0-2])\.\d{2}$/.test(value))
    return "Expiry must be in MM/YY format.";
  return null;
}
```

7.3 Creating a reservation and payment record (server)

```
// routes/reservations.js
router.post("/", requireLogin, (req, res) => {
  const { locationId, vehicle, start, end, cardLast4 } = req.body;
  const loc = db.locations.find(l => l.id === locationId);
  if (!loc || loc.availableSlots < 1)
    return res.status(409).json({ error: "No slots available" });

  const hours = Math.ceil((new Date(end) - new Date(start)) / 36e5);
  const reservation = {
    id: Date.now().toString(), userId: req.user.id, locationId, vehicle,
    start, end, status: "active",
    payment: { amount: hours * loc.pricePerHour, card: cardLast4,
      txn: "TXN-" + Math.random().toString(36).slice(2, 10).toUpperCase() },
  };
  loc.availableSlots -= 1;
  db.reservations.push(reservation);
  res.status(201).json(reservation);
});
```

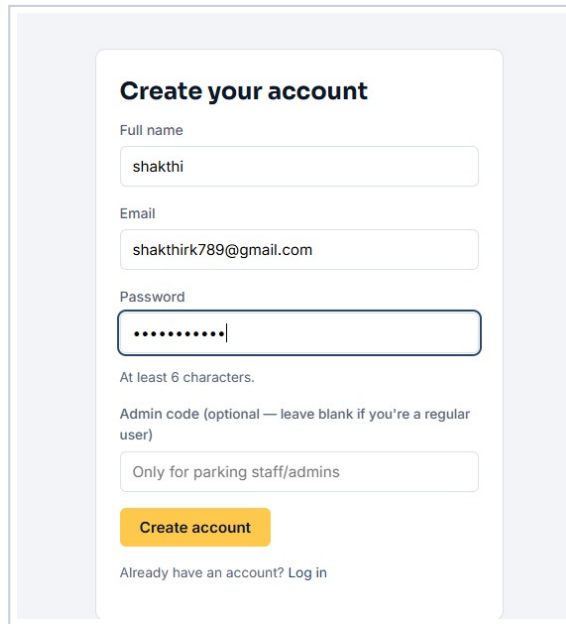
7.4 Cancelling a reservation with simulated refund (server)

```
// routes/reservations.js
router.post("/:id/cancel", requireLogin, (req, res) => {
  const r = db.reservations.find(x => x.id === req.params.id && x.userId === req.user.id);
  if (!r || r.status !== "active")
    return res.status(404).json({ error: "Reservation not found" });

  r.status = "cancelled";
  r.payment.refunded = true; // simulated refund
  db.locations.find(l => l.id === r.locationId).availableSlots += 1;
  res.json(r);
});
```

8. Screenshots

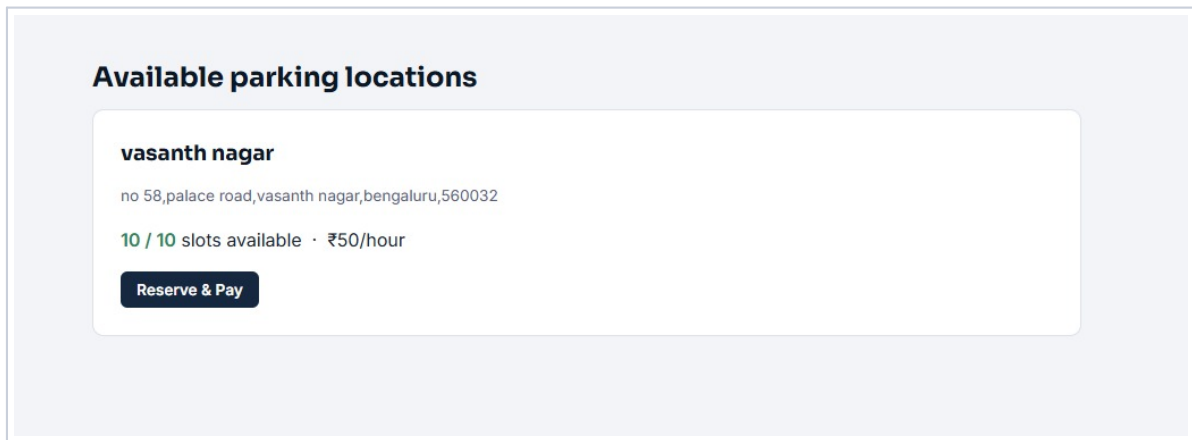
8.1 Authentication



The screenshot shows a 'Create your account' form. It includes input fields for 'Full name' (containing 'shakthi'), 'Email' (containing 'shakthirk789@gmail.com'), and 'Password' (containing masked characters). Below the password field is a note 'At least 6 characters.' and an 'Admin code (optional — leave blank if you're a regular user)' field with the placeholder text 'Only for parking staff/admins'. A yellow 'Create account' button is at the bottom, followed by a link 'Already have an account? Log in'.

Figure 1: Create-account page with optional admin code

8.2 Browse and Reserve



The screenshot shows a card titled 'Available parking locations'. Inside the card, the location 'vasanth nagar' is listed with the address 'no 58,palace road,vasanth nagar,bengaluru,560032'. It also displays '10 / 10 slots available · ₹50/hour' and a dark blue 'Reserve & Pay' button.

Figure 2: Available parking locations with live slot count and price

[← Back to locations](#)

Checkout

vasanth nagar

no 58,palace road,vasanth nagar,bengaluru,560032

Vehicle number

Start time

End time

Duration 2 hours

Rate ₹50 / hour

Total to pay ₹100.00

Payment details

.....

CARDHOLDER NAME MM/YY

Name on card

Card number

Expiry (MM/YY)

CVV

Figure 3: Checkout page with duration, rate, total and card details

SANJANA M

90/89

Expiry must be in MM/YY format.

Name on card

Card number

Expiry (MM/YY)

CVV

This is a simulated checkout for demo purposes — no real payment is processed and no card data is stored.

Pay ₹100.00

Figure 4: Card validation error shown for an invalid expiry format

8.3 Payment Confirmation

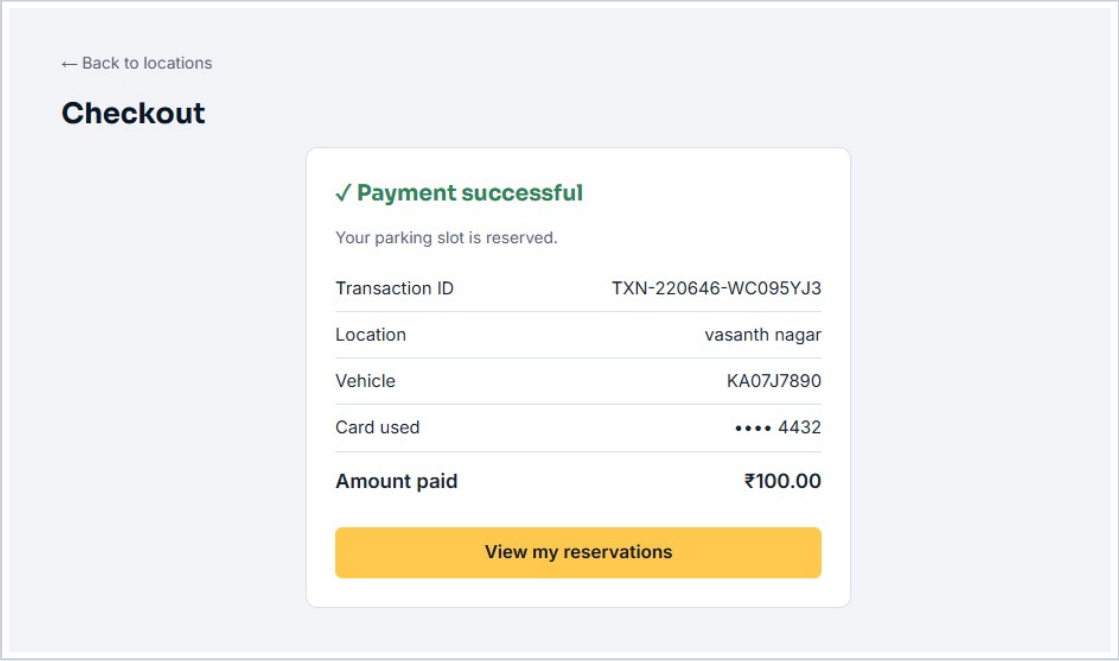


Figure 5: Payment successful receipt with transaction ID

8.4 My Reservations

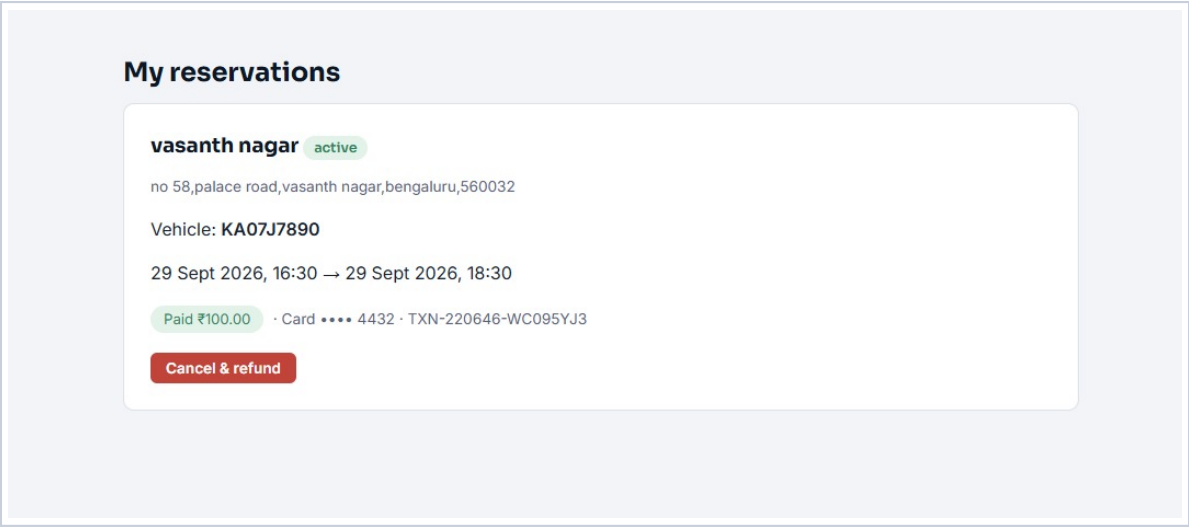


Figure 6: Active reservation with paid amount and Cancel & refund option

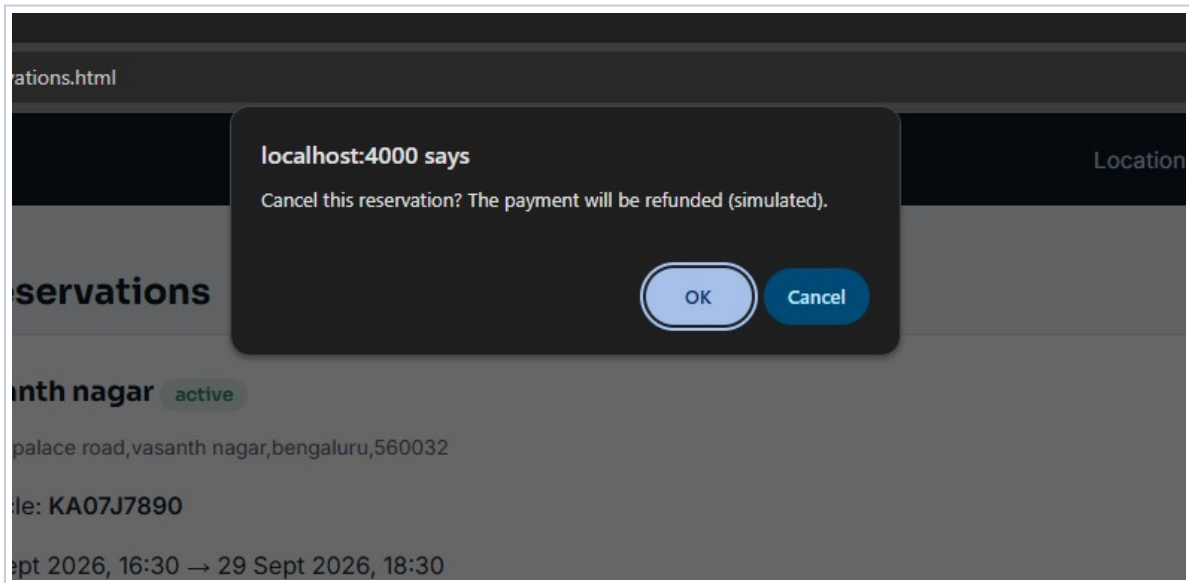


Figure 7: Confirmation dialog before cancelling a reservation

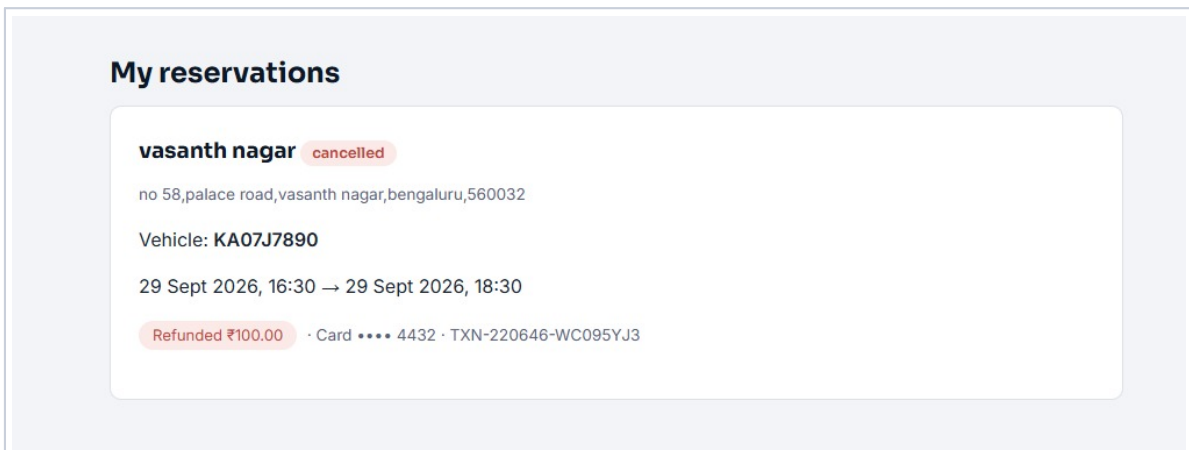


Figure 8: Cancelled reservation showing refunded amount

8.5 Admin Dashboard

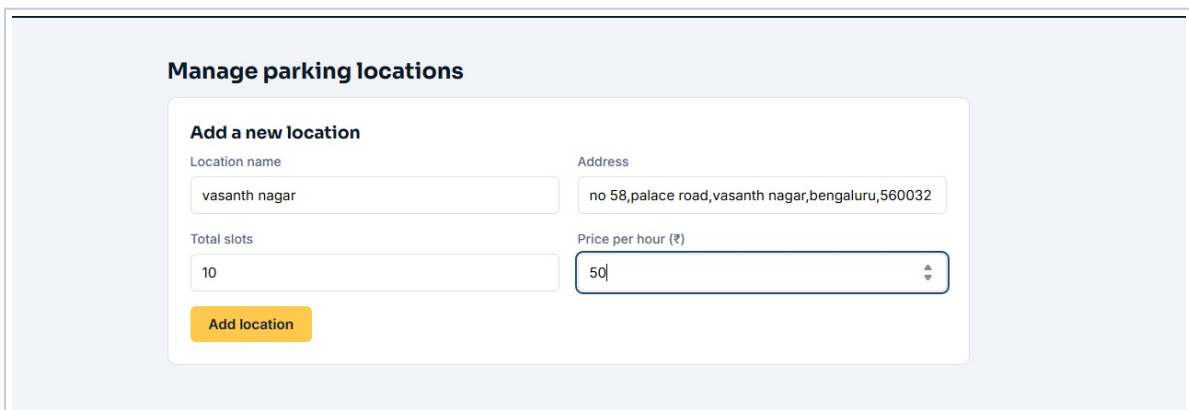


Figure 9: Admin form to add a new parking location

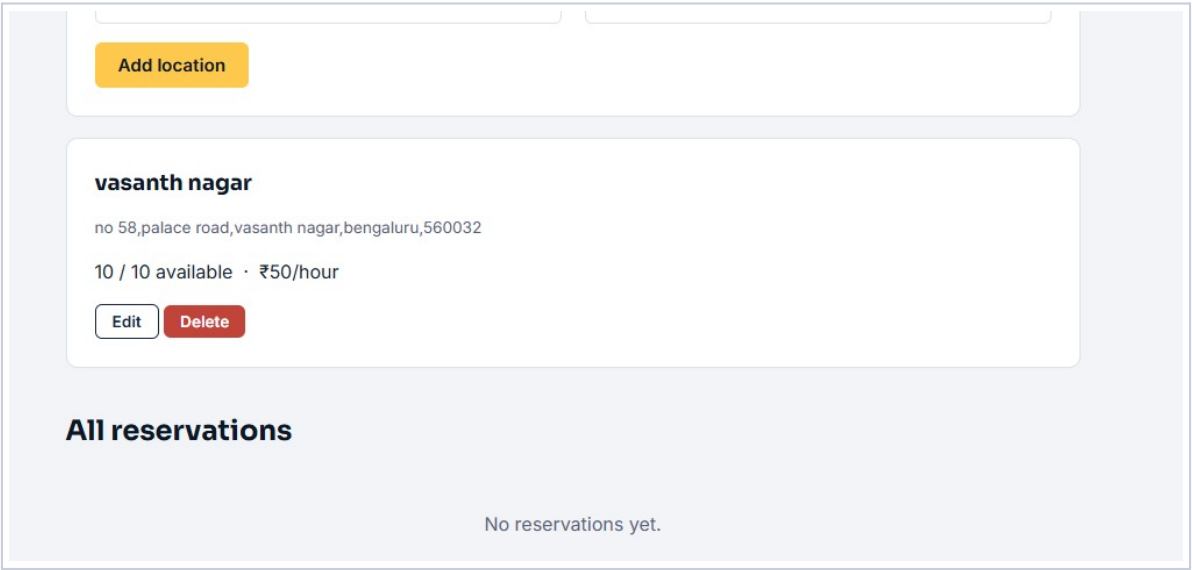


Figure 10: Managed location list with Edit / Delete and the All reservations section

9. Conclusion

The Smart Parking Reservation System successfully demonstrates a complete online parking workflow: users can discover available slots, reserve and pay with validated input, and cancel with a refund, while administrators can manage locations, capacity and pricing from a dedicated dashboard. Role-based access keeps admin functions separate from regular users, and the interface gives clear feedback at every step through status badges, confirmation dialogs and receipts.

Future Enhancements

- Integrate a real payment gateway (e.g. Razorpay) in place of the simulated checkout.
- Add a map view and live slot sensors for real-time occupancy.
- Send email/SMS confirmations and reminders before a booking ends.
- Prevent overlapping bookings per slot and add a full revenue analytics view for admins.
- Deploy the application on a cloud platform with a production database.